

CFD架构文档

1.CFD架构概况

1.1 架构愿景 (Architecture Vision)

- 项目目标：构建一个符合监管要求、高性能、可扩展的 CFD 交易平台，支持多种底层资产（如外汇、股票、商品、指数）的差价合约交易。
- 业务目标：提高市场份额、降低运营成本、确保交易的合规性和透明度。
- 主要干系人：业务发起人、合规团队、交易员、IT 开发团队、数据分析师。
- 约束与限制：遵循当地金融监管规定（如杠杆限制、保证金要求）、确保低延迟的交易执行、高可用性和数据安全性。

1.2 业务架构 (Business Architecture) (待确定)

- 业务模型：采用经纪商模型，撮合或处理客户订单，并提供实时市场数据和风险管理。
- 业务能力：
 - 客户管理：账户开立、KYC/AML 验证、资金存取。
 - 产品管理：定义和配置可交易的 CFD 产品、底层资产映射。
 - 交易执行：订单录入、路由、撮合、执行确认。
 - 风险管理：实时保证金计算、强制平仓 (Margin Call)、止损/止盈管理。
 - 结算与报告：每日结算、客户报表生成、监管报告提交。
- 价值流：从客户注册 -> 资金存入 -> 交易开仓/平仓 -> 盈亏结算 -> 资金取出。

1.3 信息系统架构 (Information Systems Architecture)

1.3.1 数据架构 (Data Architecture)

- 核心数据实体：
 - 客户 (Customer)：包含个人信息、合规状态、账户详情。
 - 账户 (Account)：包含余额、保证金水平、杠杆率。
 - 订单 (Order)：包含类型（市价、限价）、方向（买/卖）、数量、价格。
 - 头寸 (Position)：包含开仓价、当前市值、未实现盈亏。
 - 交易记录 (Transaction Log)：包含所有已执行交易的详细信息、时间戳。

- 市场数据 (Market Data): 包含实时报价、历史价格数据。
- 数据流: 市场数据流入 -> 交易系统 -> 数据库存储 -> 风险管理系统实时读取 -> 报告系统提取数据。
- 数据治理: 定义数据所有权、数据质量标准、访问控制和审计跟踪流程。

1.3.1.1 用户注册--》入金流程

1.3.1.1.1 用户注册

- 触发事件: 用户访问CFD交易平台网站或应用, 点击注册按钮。
- 数据流:
 - 用户填写注册信息 (用户名、密码、邮箱、手机号等)。
 - 平台前端对用户输入进行基本验证 (如格式检查)。
 - 验证通过后, 前端将注册信息加密并发送至后端服务器。
 - 后端服务器接收信息, 进行进一步验证 (如邮箱/手机号唯一性检查)。
 - 验证成功后, 后端服务器创建用户账户, 并生成唯一的用户ID。
 - 用户ID与基本信息存储至数据库, 同时触发欢迎邮件/短信发送至用户。

1.3.1.1.2 KYC (客户身份验证)

- 触发事件: 用户登录平台后, 进入个人中心或KYC专区提交身份验证信息。
- 数据流:
 - 用户上传身份证明文件 (如身份证、护照等) 和居住证明 (如水电费账单、银行对账单等)。
 - 前端对上传的文件进行格式和大小验证。
 - 验证通过后, 文件被加密并发送至后端服务器。
 - 后端服务器接收文件, 进行病毒扫描和恶意内容检测。
 - 检测通过后, 文件被转发至KYC审核团队或自动审核系统 (如OCR识别+人脸识别技术)。
 - 审核结果 (通过/拒绝) 及审核意见返回至后端服务器, 并更新用户KYC状态。
 - 用户接收KYC审核结果通知 (邮件/短信/站内信)。

1.3.1.1.3 测评 (风险评估)

- 触发事件: KYC审核通过后, 用户被引导至测评页面或个人中心中可主动发起测评。
- 数据流:
 - 用户填写测评问卷 (包括投资经验、风险承受能力、投资目标等)。
 - 前端对用户输入进行逻辑验证。
 - 验证通过后, 测评数据被加密并发送至后端服务器。

- 后端服务器接收数据，进行风险评估计算（如使用预定算法或模型）。
- 评估结果（风险等级、建议投资产品等）生成并存储至数据库。
- 用户接收测评结果通知，并可根据结果查看推荐的投资产品。

1.3.1.1.4 入金

- 触发事件：用户完成测评后，在交易平台选择入金操作。
- 数据流：
 - 用户选择入金方式（如银行转账、信用卡/借记卡支付、电子钱包等）。
 - 用户填写入金金额和支付信息（如卡号、有效期、CVV码等）。
 - 前端对用户输入进行验证（如金额格式、支付信息有效性等）。
 - 验证通过后，支付信息被加密并发送至支付网关或第三方支付平台。
 - 支付网关或第三方支付平台处理支付请求，并返回支付结果至后端服务器。
 - 后端服务器接收支付结果，更新用户账户余额，并生成入金记录存储至数据库。
 - 用户接收入金结果通知（邮件/短信/站内信），并可在平台查看更新后的账户余额。

1.3.1.2 用户下单流程

1.3.1.3 用户平仓流程

1.3.1.4 系统自动平仓流程

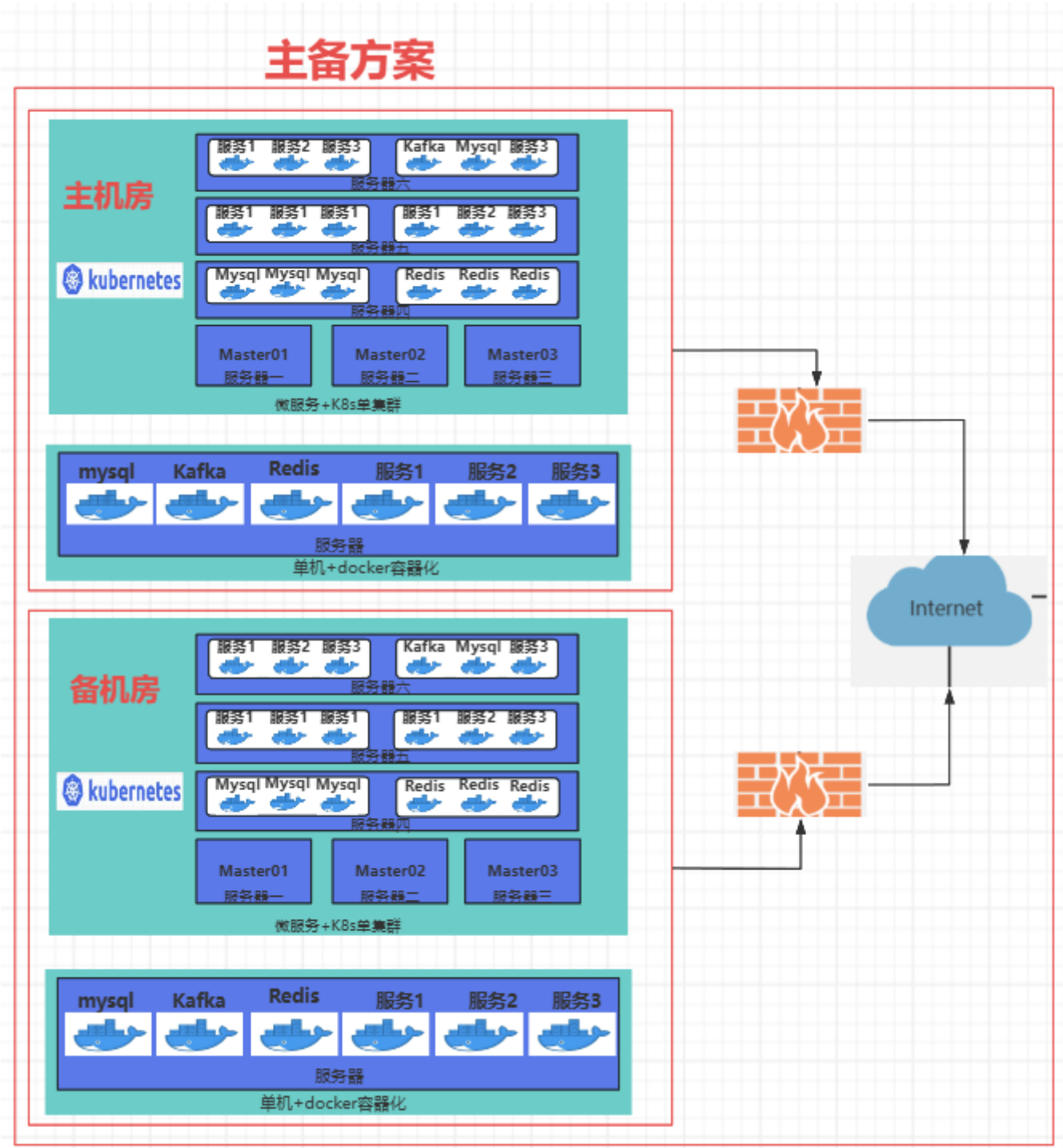
1.3.1.5 隔夜费计算流程

1.3.2 应用架构 (Application Architecture)

- 核心应用组件：
 - 客户门户 (Customer Portal): Web/移动前端，用于客户交互、账户管理、存取款。
 - 交易引擎 (Trading Engine): 核心后端服务，负责订单处理、撮合和执行。
 - 市场数据服务 (Market Data Service): 负责聚合、清洗和分发实时市场数据。
 - 风险管理系统 (Risk Management System): 实时监控头寸、计算保证金、执行强制平仓逻辑。
 - 清算引擎 (Clearing & Settlement Engine): 负责每日盈亏结算和费用计算。
 - 合规与报告服务 (Compliance & Reporting Service): 生成监管所需的交易和财务报告。

1.3.3 技术架构（Technical Architecture）

CFD（Contract for Difference）交易平台作为金融交易系统，对高可用性、可扩展性、容错性以及低延迟处理有极高的要求。本架构设计旨在通过主备模式结合Kubernetes（K8s）容器编排技术，实现交易平台的高可用部署和弹性伸缩，确保交易服务的连续性和稳定性。详见下面示意图



1.3.3.1 主备模式概述

主备模式是一种常见的高可用性解决方案，通过部署主节点和备用节点，当主节点发生故障时，备用节点能够迅速接管服务，从而最小化服务中断时间。在CFD交易平台中，我们将关键服务如交易引擎、订单管理、风控服务等部署在主备节点上，确保交易服务的连续性。

1.3.3.2 Kubernetes（K8s）部署方案

Kubernetes是一个开源的容器编排平台，用于自动化部署、扩展和管理容器化应用。在CFD交易平台中，我们采用K8s进行容器化部署，以实现以下目标：

1. 容器化应用：

- 将CFD交易平台的各个服务（用户中心服务、资产账户服务、交易引擎服务等）打包成Docker镜像，实现应用与环境的解耦。
- 通过K8s的Pod对象部署容器，确保每个服务都能独立运行且易于管理。

2. 主备节点部署：

- 在K8s集群中配置主节点和备用节点，使用K8s的StatefulSet或Deployment对象管理主备服务的部署。
- 利用K8s的Health Check机制监控主节点的健康状态，一旦检测到主节点故障，K8s将自动将服务切换到备用节点。

3. 服务发现与负载均衡：

- 使用K8s的Service对象实现服务发现，确保客户端能够动态发现并连接到可用的服务实例。
- 结合Ingress控制器实现外部流量到内部服务的负载均衡，提高系统的并发处理能力和可用性。

4. 弹性伸缩：

- 利用K8s的Horizontal Pod Autoscaler（HPA）根据CPU利用率等指标自动调整Pod的数量，实现服务的弹性伸缩。
- 在交易高峰期，HPA将自动增加Pod实例以应对高并发请求；在低峰期则减少实例以节省资源。

5. 持久化存储：

- 对于需要持久化存储的数据（如交易记录、用户信息等），使用K8s的PersistentVolume（PV）和PersistentVolumeClaim（PVC）机制实现数据的持久化存储和访问。
- 结合分布式存储系统（如Ceph、NFS等）提供高可用和可扩展的存储解决方案。

6. 监控与日志：

- 集成Prometheus和Grafana等监控工具，实时监控K8s集群和CFD交易平台的运行状态和性能指标。
- 使用ELK（Elasticsearch、Logstash、Kibana）或EFK（Elasticsearch、Fluentd、Kibana）等日志管理方案，集中收集、存储和分析交易平台的日志信息。

1.3.3.3 高可用性与容错性设计

1. 多副本部署：

- 对于非关键服务，可以采用多副本部署方式，提高服务的可用性和容错性。
- 通过K8s的Deployment对象轻松实现多副本部署和管理。

2. 故障转移与恢复：

- 当主节点发生故障时，K8s将自动触发故障转移机制，将服务切换到备用节点。
- 结合备份和恢复策略，确保在故障发生后能够迅速恢复数据和服务。

3. 网络与安全：

- 使用K8s的Network Policy对象实现容器间的网络隔离和访问控制。
- 结合TLS/SSL加密技术保障交易数据在传输过程中的安全性。

1.3.3.4 实施步骤

1. 环境准备：

- 搭建K8s集群，包括主节点和多个工作节点。
- 配置存储系统，确保数据持久化存储的可用性。

2. 容器化应用打包：

- 将CFD交易平台的各个服务打包成Docker镜像，并推送到镜像仓库。

3. K8s部署配置：

- 编写K8s的YAML配置文件，定义各个服务的Deployment、Service、Ingress等对象。
- 配置HPA实现服务的弹性伸缩。

4. 监控与日志配置：

- 集成Prometheus、Grafana等监控工具，配置监控指标和报警规则。
- 部署ELK或EFK日志管理方案，集中收集和分析日志信息。

5. 测试与验证：

- 进行功能测试、性能测试、压力测试等，确保交易平台的稳定性和可靠性。
- 验证主备切换、故障转移等高可用性功能的有效性。

2. 微服务架构

2.1 整体架构概览

CFD交易平台采用微服务架构，将系统划分为多个独立的服务模块，各模块之间通过轻量级通信协议（如HTTP/gRPC）交互，并通过统一网关对外暴露接口。系统整体架构如下所示：

2.1.1 用户中心服务（User Service）

- 功能：负责用户注册、登录、身份验证、KYC认证等。
- 特点：支持OAuth2/JWT认证机制，保障用户信息安全。

2.1.2 资产账户服务（Account Service）

- 功能：管理用户的资金账户、持仓信息、出入金记录。
- 特点：支持多币种账户和实时余额更新。

2.1.3 交易引擎服务 (Trading Engine Service)

- 功能：核心交易匹配逻辑，处理买卖订单撮合。
- 特点：高性能低延迟，支持市价单、限价单等多种订单类型。

2.1.4 行情数据服务 (Market Data Service)

- 功能：提供实时和历史行情数据，包括价格、成交量等。
- 特点：支持WebSocket推送实时行情，对接第三方行情源。

2.1.5 订单管理服务 (Order Management Service)

- 功能：管理用户提交的所有订单状态（挂单、成交、撤单等）。
- 特点：支持订单生命周期全流程追踪。

2.1.6 风控服务 (Risk Control Service)

- 功能：实时监控交易行为，防范异常交易、恶意刷单等风险。
- 特点：支持动态风控策略配置和自动熔断机制。

2.1.7 清算结算服务 (Clearing & Settlement Service)

- 功能：每日结算用户账户损益，处理资金划转。
- 特点：支持定时任务与手动清算操作。

2.1.8 通知服务 (Notification Service)

- 功能：向用户发送交易确认、风控提醒等消息。
- 特点：支持短信、邮件、站内信、消息推送等多种通知方式。

2.1.9 日志与监控服务 (Logging & Monitoring Service)

- 功能：收集服务运行日志，监控系统健康状态。
- 特点：集成Prometheus + Grafana进行可视化监控。

2.1.10 报表服务 (Reporting Services)

- 功能：生成报表，供客户、运营、公司高层使用。
- 特点：统一收集日志、同步数据库数据，异步存算一体。

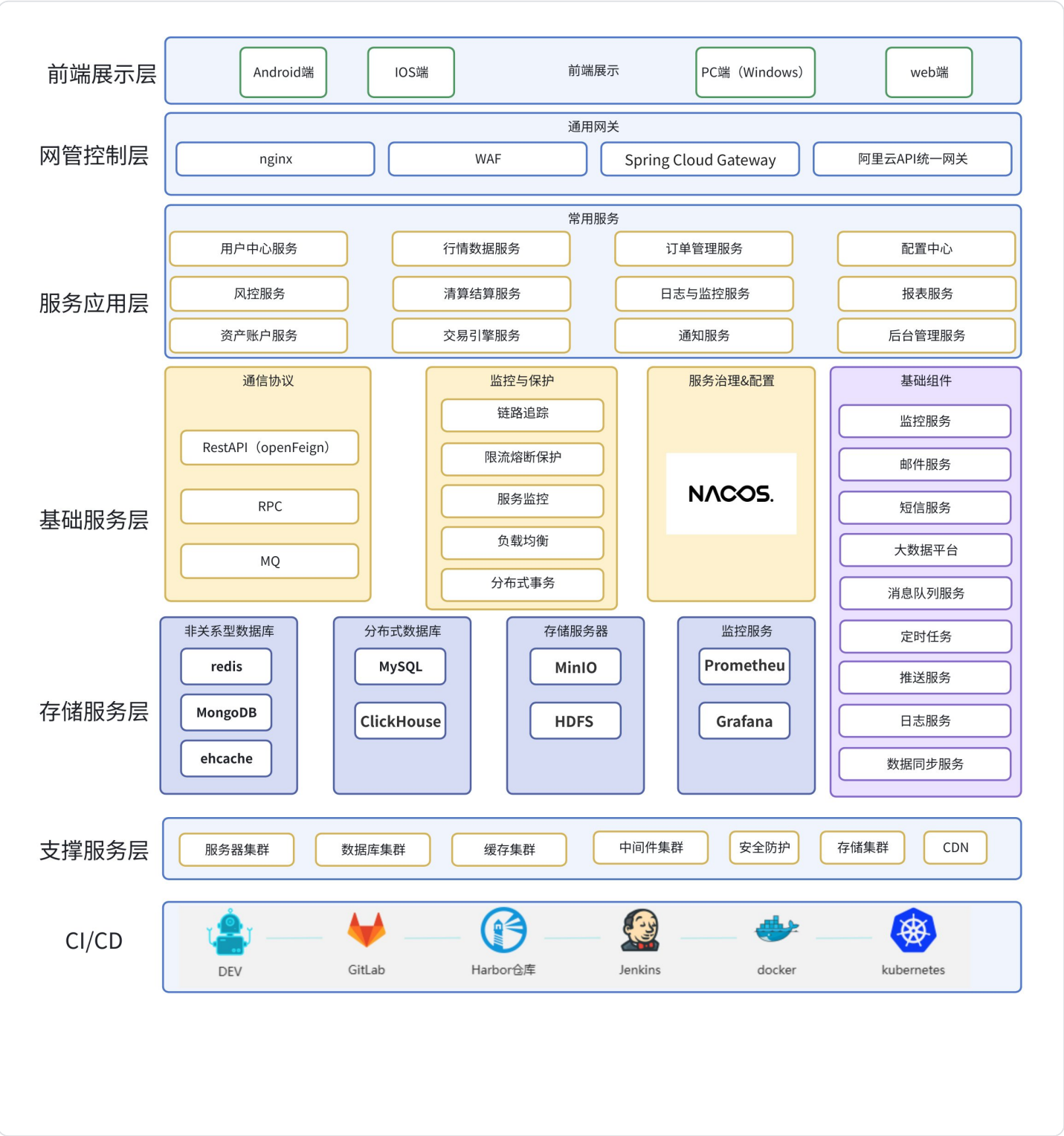
2.1.11 配置中心 (Config Service)

- 功能：集中管理服务配置。
- 特点：所有的配置集中管理、品种、牌照、国家、点差、杠杆率等。

2.1.12 后台管理服务(Backend Management Service)

- 功能：公司用户内部使用，包含用户管理、内容管理、订单和交易管理、客户支持和服务。
- 特点：提高业务运营效率、确保数据安全、促进决策制定的重要工具。

2.2 微服务架构图



3. 服务模块详细划分

3.1 用户中心服务（User Service）

用户中心服务是CFD平台的核心基础服务，负责处理所有与用户相关的功能，包括用户生命周期管理、身份认证、权限控制和合规验证。该服务采用微服务架构设计，确保高可用性、安全性和可扩展性。

- 功能：负责用户注册、登录、身份验证、KYC认证等。
- 特点：支持OAuth2/JWT认证机制，保障用户信息安全。

3.1.1 核心功能模块

3.1.1.1 用户管理模块

- 用户注册与账户创建
- 用户信息维护（个人资料、联系方式等）
- 用户状态管理（激活、冻结、注销）
- 用户等级与权限体系

3.1.1.2 认证授权模块

- OAuth2.0协议实现
- JWT Token生成与验证
- 多因素认证（MFA）支持
- Session管理与单点登录（SSO）

3.1.1.3 KYC/AML合规模块

- 身份验证（身份证、护照等）
- 地址验证与居住证明
- 政治敏感人员（PEP）筛查
- 反洗钱（AML）风险评估

3.1.1.4 安全防护模块

- 密码策略与加密存储
- 异常登录检测与防护
- IP白名单/黑名单管理
- 操作日志审计追踪

3.1.2 技术架构设计

3.1.2.1 服务架构图

textCopy Code



3.1.2.2 数据存储设计

- 主数据库：MySQL 8.0集群，存储用户核心信息
- 缓存层：Redis 6.0集群，缓存Token和会话信息
- 日志存储：Elasticsearch+Kibana，存储操作审计日志
- 文件存储：MinIO分布式对象存储，存储KYC文档

3.1.2.3 安全设计

- 数据传输：全站HTTPS加密，TLS 1.3协议
- 数据存储：敏感信息AES-256加密，密码BCrypt哈希
- API安全：OAuth2 + JWT，支持Scope权限控制
- 访问控制：基于RBAC的权限模型

3.1.2.4 API接口设计（参考，需要根据业务详细设计）

3.1.2.4.1 认证接口

- POST /auth/register - 用户注册
- POST /auth/login - 用户登录
- POST /auth/refresh - Token刷新
- POST /auth/logout - 用户登出

3.1.2.4.2 用户管理接口

- GET /users/{id} - 获取用户信息
- PUT /users/{id} - 更新用户信息

- POST /users/{id}/password - 修改密码
- DELETE /users/{id} - 注销账户

3.1.2.4.3 KYC认证接口

- POST /kyc/applications - 提交认证申请
- GET /kyc/applications/{id} - 查询认证状态
- PUT /kyc/applications/{id} - 更新认证信息

3.1.3 部署架构

- 容器化部署：Docker + Kubernetes
- 负载均衡：Nginx Ingress Controller
- 自动扩缩容：HPA基于CPU/内存指标
- 监控告警：Prometheus + Grafana
- 日志收集：ELK Stack集中日志管理

3.2 资产账户服务 (Account Service)

资金账户服务是CFD平台的核心交易服务，负责管理用户的资金账户、持仓信息和资金流水记录。该服务采用高可用、高并发的微服务架构设计，确保资金数据的一致性和实时性。

- 功能：管理用户的资金账户、持仓信息、出入金记录。
- 特点：支持多币种账户和实时余额更新。

3.2.1 核心功能模块

3.2.1.1 账户管理模块

- 多币种账户创建与维护
- 账户余额实时计算与更新
- 账户状态管理（正常、冻结、注销）
- 账户权限与限制设置

3.2.1.2 持仓管理模块

- 持仓头寸创建与更新
- 实时盈亏计算
- 保证金监控与强平预警
- 持仓历史记录追踪

3.2.1.3 资金流水模块

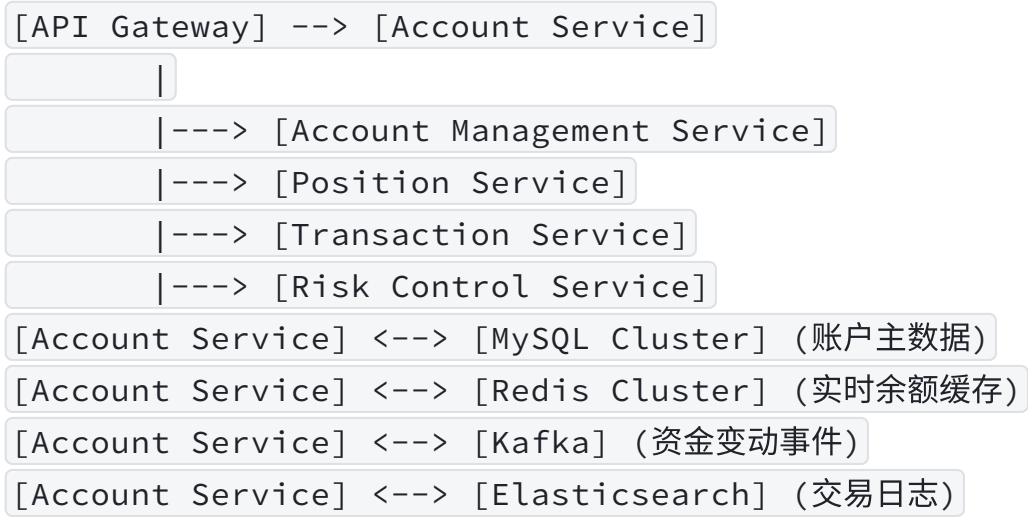
- 入金/出金处理
- 交易手续费计算与扣除
- 资金流水记录生成
- 账户余额变动审计

3.2.1.4 风险控制模块

- 保证金率实时监控
- 强制平仓触发机制
- 账户风险等级评估
- 异常交易行为检测

3.2.2 技术架构设计

3.2.2.1 服务架构图



3.2.2.2 数据存储设计

- 主数据库：MySQL 8.0集群，存储账户、持仓和交易记录
- 缓存层：Redis 6.0集群，缓存账户余额和持仓信息
- 日志存储：Elasticsearch，存储资金流水和操作日志
- 消息队列：Kafka，处理账户变动事件通知

3.2.2.3 安全设计

- 数据传输：全站HTTPS加密，TLS 1.3协议
- 数据存储：敏感信息AES-256加密
- API安全：JWT Token验证，支持Scope权限控制

- 操作审计：所有资金操作实现不可篡改审计追踪

3.2.2.4 API接口设计（参考，需要根据业务详细设计）

3.2.2.4.1 账户管理接口

- GET /accounts/{accountId} - 获取账户信息
- PUT /accounts/{accountId} - 更新账户设置
- GET /accounts/{accountId}/balance - 获取账户余额

3.2.2.4.2 持仓管理接口

- GET /positions - 查询持仓列表
- GET /positions/{positionId} - 获取持仓详情
- POST /positions - 创建新持仓
- DELETE /positions/{positionId} - 平仓操作

3.2.2.4.3 资金流水接口

- GET /transactions - 查询资金流水
- POST /transactions/deposit - 入金操作
- POST /transactions/withdraw - 出金操作
- GET /transactions/{transactionId} - 获取交易详情

3.2.2.4.4 风险控制接口

- GET /risk/account/{accountId} - 获取账户风险状态
- POST /risk/margin/call - 发送追加保证金通知
- POST /risk/liquidation - 执行强制平仓

3.2.3 部署架构

- 容器化部署：Docker + Kubernetes
- 负载均衡：Nginx Ingress Controller
- 自动扩缩容：HPA基于CPU/内存指标
- 监控告警：Prometheus + Grafana
- 日志收集：ELK Stack集中日志管理

3.3 交易引擎服务（Trading Engine Service）

交易引擎服务作为金融交易系统的核心组件，肩负着执行核心交易匹配逻辑以及处理买卖订单的关键任务。其核心目标是在复杂多变的市场环境中，以高性能、低延迟的方式准确无误地完成订单匹配与

交易执行，同时支持市价单、限价单等多种常见的订单类型，为交易系统的高效稳定运行提供坚实保障。

- 功能：核心交易匹配逻辑，处理买卖订单。
- 特点：高性能低延迟，支持市价单、限价单等多种订单类型。

3.3.1 核心功能模块

3.3.1.1 订单接收与分类

- 接收来自不同渠道（如交易前端、API接口等）的各类订单请求，包括市价单、限价单等。
- 对订单进行初步分类和整理，依据订单类型将其引导至相应的处理流程。

3.3.1.2 核心交易匹配

- 运用先进的匹配算法，依据价格优先、时间优先等原则，对买卖订单进行精准匹配。
- 针对市价单，以当前市场最优价格迅速完成匹配；对于限价单，则在市场价格达到指定限价时进行匹配。

3.3.1.3 交易执行与记录

- 成功匹配后，立即执行交易，更新相关账户的资金和持仓信息。
- 详细记录每一笔交易的交易时间、交易价格、交易数量等关键信息，形成完整的交易日志。

3.3.2 技术架构设计

3.3.2.1 服务架构图

[外部系统] --订单请求--> [API网关] --解析与路由--> [交易引擎核心服务]

[交易引擎核心服务] 包含以下子模块：

[订单接收模块] --> [订单分类模块] --> [交易匹配模块（市价单匹配子模块、限价单匹配子模块）] --> [交易执行模块] --> [交易记录模块]

[交易引擎核心服务] 与外部组件的交互：

[交易引擎核心服务] <--> [数据库（存储订单、交易记录等数据）]

[交易引擎核心服务] <--> [缓存（用于缓存实时行情、订单状态等信息）]

[交易引擎核心服务] <--> [消息队列（用于模块间通信和异步处理）]

3.3.2.2 子模块设计

1. 订单接收模块

- 功能：负责接收来自API网关的订单请求，并进行初步的合法性检查，如订单格式是否正确、必要字段是否缺失等。

- 实现方式：采用高性能的网络通信框架，如Netty，以异步非阻塞的方式处理订单请求，提高系统的并发处理能力。

2. 订单分类模块

- 功能：根据订单的类型（市价单、限价单等），将订单分发至相应的匹配子模块进行处理。
- 实现方式：通过定义清晰的订单类型标识和分类规则，利用条件判断或策略模式，将订单准确路由到对应的处理模块。

3. 交易匹配模块

- 市价单匹配子模块
 - 功能：针对市价单，以当前市场最优价格迅速寻找合适的手方订单进行匹配。
 - 实现方式：实时监控市场行情和订单簿，运用快速查找算法，在买卖盘口中找到最优价格的订单进行匹配。
- 限价单匹配子模块
 - 功能：对于限价单，当市场价格达到或优于订单指定的限价时，进行订单匹配。
 - 实现方式：将限价单存储在相应的价格队列中，定期检查市场价格，当市场价格满足条件时，触发匹配操作。

4. 交易执行模块

- 功能：在订单匹配成功后，执行实际的交易操作，更新买卖双方的账户资金和持仓信息。
- 实现方式：与账户管理系统进行交互，采用事务处理机制，确保账户数据的一致性和准确性。在更新过程中，如果出现异常情况，能够及时回滚操作，保证系统的稳定性。

5. 交易记录模块

- 功能：详细记录每一笔交易的相关信息，包括交易时间、交易产品、成交价格、成交数量、买卖方向等，并将记录存储到数据库中。
- 实现方式：在交易执行完成后，生成标准化的交易记录数据，通过数据库连接池将记录插入到数据库表中。同时，可以将重要的交易记录缓存到缓存系统中，以便快速查询和展示。

3.3.2.3 数据存储设计（参考）

- 数据库选型：选用高性能的关系型数据库（如Oracle、MySQL集群）来存储订单、交易记录等结构化数据，确保数据的完整性和一致性。同时，结合使用时序数据库（如InfluxDB）来存储市场行情数据，以满足对时间序列数据的高效读写需求。
- 数据表设计
 - 订单表：包含订单ID、用户ID、交易产品、订单类型、价格、数量、状态、提交时间等字段，用于记录所有订单的详细信息。
 - 交易记录表：存储交易ID、订单ID、成交价格、成交数量、买卖方向、成交时间等信息，记录每一笔成交的具体情况。

- 市场行情表：保存交易产品、最新成交价、成交量、买卖盘口数据等，实时反映市场动态。

3.3.2.4 性能优化策略

1. 缓存优化：充分利用缓存技术，将频繁访问的数据（如实时行情、热门订单状态等）缓存到内存中，减少对数据库的访问次数，提高数据读取速度。
2. 并行处理：对于一些可以并行执行的任务，如订单的合法性检查、交易记录的存储等，采用多线程或分布式处理技术，提高系统的整体处理能力。
3. 算法优化：不断优化交易匹配算法，采用更高效的数据结构和查找算法，减少匹配时间，提高匹配效率。

3.3.2.5 安全设计

1. 数据安全：对敏感数据（如用户账户信息、交易密码等）进行加密存储和传输，采用SSL/TLS协议保障数据在网络传输过程中的安全性。
2. 系统可靠性：采用冗余部署和故障转移机制，确保在部分服务器出现故障时，交易引擎服务仍能正常运行。同时，建立完善的备份恢复策略，定期对数据进行备份，以防止数据丢失。

通过以上架构设计，交易引擎服务能够在高性能、低延迟的前提下，准确高效地处理各类买卖订单，支持多种订单类型，为金融交易系统提供可靠的核心交易支持。

3.3.3 部署架构

- 容器化部署：Docker + Kubernetes
- 负载均衡：Nginx Ingress Controller
- 自动扩缩容：HPA基于CPU/内存指标
- 监控告警：Prometheus + Grafana
- 日志收集：ELK Stack集中日志管理

3.4 行情数据服务（Market Data Service）

- 功能：提供实时和历史行情数据，包括价格、成交量等。
- 特点：支持WebSocket推送实时行情，对接第三方行情源。

3.5 订单管理服务（Order Management Service）

- 功能：管理用户提交的所有订单状态（挂单、成交、撤单等）。
- 特点：支持订单生命周期全流程追踪。

3.6 风控服务（Risk Control Service）

- 功能：实时监控交易行为，防范异常交易、恶意刷单等风险。

- 特点：支持动态风控策略配置和自动熔断机制。

3.7 清算结算服务（Clearing & Settlement Service）

- 功能：每日结算用户账户损益，处理资金划转。
- 特点：支持定时任务与手动清算操作。

3.8 通知服务（Notification Service）

- 功能：向用户发送交易确认、风控提醒等消息。
- 特点：支持短信、邮件、站内信、消息推送等多种通知方式。

3.9 日志与监控服务（Logging & Monitoring Service）

- 功能：收集服务运行日志，监控系统健康状况。
- 特点：集成Prometheus + Grafana进行可视化监控。

3.10 报表服务（Reporting Services）

- 功能：生成报表，供客户、运营、。
- 特点：统一收集日志、同步数据库数据，异步存算一体。

3.11 配置中心（Config Service）

- 功能：集中管理服务配置。
- 特点：所有的配置集中管理、品种、牌照、国家、点差、杠杆率等等。

3.12 后台管理服务(Backend Management Service)

- 功能：公司用户内部使用，包含用户管理、内容管理、订单和交易管理、客户支持和服务。
- 特点：提高业务运营效率、确保数据安全、促进决策制定的重要工具。

4. 部署架构建议

- 所有服务基于容器化部署（Docker + Kubernetes）
- 使用服务注册与发现机制（如Consul或Eureka）
- 各服务之间通过内部REST/gRPC通信
- 数据库按服务拆分，避免共享数据库
- 引入消息队列（如Kafka、RocketMQ）用于异步处理订单、通知等事件